

Complex numbers: rotate 90°: i = e^{i\pi/2} = cos(\pi/2) + i sin(\pi/2) = 0 + i*1 = i. Labeled in unit circle. e^{i\theta} = cos(\theta) + i sin(\theta). e^{i\pi} = -1, e^{i2\pi} = 1.

Vector Space: Set of vectors closed under addition and scalar multiplication. Null Space: Set of vectors v such that Av=0. Affine Space: Set of points of the form p + Span(S).

Linear Functions: f(x) = ax + b. Kernel: f^{-1}(0). Rank-Nullity Theorem: rank(A) + nullity(A) = n.

Matrix Operations: Inverse, Determinant, Eigenvalues. LU Factorization: A = LU. Gaussian Elimination: Reorder A to be in echelon form.

Orthogonalization: Gram-Schmidt process. QR Factorization: A = QR. SVD: A = U\Sigma V^T. Eigenvalues and Eigenvectors: A\vec{v} = \lambda\vec{v}.

Similarity: A and B are similar if B = S^{-1}AS. Diagonalizability: A is diagonalizable if it has a basis of eigenvectors. Idempotent: A^2 = A.

Stochastic Matrices: Rows sum to 1. Markov Chains: Discrete-time stochastic process.

Linear Algebra Applications: Computer graphics, physics, economics, data science.

ISAK ELI HAUSSON

Linear Functions

Proposition: Let M be $\mathbb{R} \times \mathbb{C}$ matrix and suppose $f: \mathbb{R} \rightarrow \mathbb{R}$ is defined $f(x) = M \cdot x$ then f is a linear function

Proof:
we showed that $M \cdot (\alpha v) = \alpha M \cdot v$
this proves f satisfies **L1**
we showed that $M \cdot (x+y) = M \cdot x + M \cdot y$
this proves f satisfies **L2**

Matrix-Matrix Mult

Proof:
column j of AB is $A \cdot (\text{column } j \text{ of } B)$
For any x -vector $\bar{x} = [x_1, \dots, x_n]$
 $g(x) = B \cdot \bar{x} = x_1 b_1 + \dots + x_n b_n$

Therefore:
 $f(g(x)) = f(x_1 b_1 + \dots + x_n b_n)$ by **L1** and **L2**
 $= x_1(f(b_1)) + \dots + x_n(f(b_n))$
 $= x_1(A \cdot b_1) + \dots + x_n(A \cdot b_n)$
 $= x_1(\text{col } 1 \text{ of } AB) + \dots + x_n(\text{col } n \text{ of } AB)$
 $= (AB) \cdot \bar{x} = h(\bar{x})$

Linear-Dependence Lemma

Let $v_1, \dots, v_n$ be vecs. a vector v_i is in span of other vecs iff zero vec can be written as linear combination of $v_1, \dots, v_n$ in which the coefficient of v_i is nonzero

Unique Representation vis Basis

Proof: v in V . The vecs $e_1, \dots, e_n$ span V , so there is at least one representation of v in terms of basis-vectors
Suppose there are 2 representations
 $v = \alpha_1 e_1 + \dots + \alpha_n e_n = \beta_1 e_1 + \dots + \beta_n e_n$ we get the zero vec by subtracting one from the other
 $0 = \alpha_1 e_1 + \dots + \alpha_n e_n - (\beta_1 e_1 + \dots + \beta_n e_n)$
 $= (\alpha_1 - \beta_1) e_1 + \dots + (\alpha_n - \beta_n) e_n$
Since vecs $e_1, \dots, e_n$ are linearly independent, $\alpha_1 - \beta_1, \dots, \alpha_n - \beta_n$ must be zero
the 2 representations are the same

Simplified Exchange Lemma

There is vec $\bar{w}$ in S such $\text{Span}(S \cup \{\bar{w}\} - \{\bar{w}_3\}) = \text{Span } S$
proof: $S = \{v_1, \dots, v_n\}$, since $\bar{w}$ is in span S
 $\bar{w} = \alpha_1 v_1 + \dots + \alpha_n v_n$ By superfluous-Vector Lemma,
 $\text{Span}(S \cup \{\bar{w}\}) = \text{Span } S$. Since $\bar{w}$ is non-zero, at least one coefficient is non-zero
 $\bar{w} = \alpha_1 v_1 + \dots + \alpha_i v_i + \dots + \alpha_n v_n = \alpha_i v_i + \dots$
Divide through by α_i
 $\frac{\bar{w}}{\alpha_i} = \frac{\alpha_1}{\alpha_i} v_1 + \dots + \frac{\alpha_i}{\alpha_i} v_i + \dots + \frac{\alpha_n}{\alpha_i} v_n = v_i + \dots$
By superfluous-Vector Lemma,
 $\text{Span}(S \cup \{\bar{w}\}) = \text{Span}(S \cup \{\bar{w}\} - \{v_i\})$

Superfluous-Vector Lemma

S and $\bar{w}$ in S : linear combination of the other vecs in S the $\text{Span}(S - \{\bar{w}\}) = \text{Span } S$
Proof: Suppose $v_n = \alpha_1 v_1 + \dots + \alpha_n v_n$. To show every vecs in $\text{Span } S$ is also in $\text{Span}(S - \{v_n\})$
Every vec $\bar{v}$ in $\text{Span } S$: $\bar{v} = \beta_1 v_1 + \dots + \beta_n v_n$
Substituting for v_n we get
 $\bar{v} = \beta_1 v_1 + \dots + \beta_n (\alpha_1 v_1 + \dots + \alpha_n v_n)$
 $= (\beta_1 + \beta_n \alpha_1) v_1 + \dots + (\beta_n + \beta_n \alpha_n) v_n$
which shows that any vec in S can be written as linear combination of vecs in $S - \{v_n\}$ and therefore in $\text{Span}(S - \{v_n\})$

Morphing Lemma

$B = \{b_1, \dots, b_n\}$, $S_0 = S$. Prove by induction on $k \leq n$ that there is generating set S_k of spans that has $b_1, \dots, b_k$ and size $|S_k|$.
Base case: $k=0$ is trivial
To go from S_{k-1} to S_k use Exchange Lemma
 $A_k = \{b_1, \dots, b_{k-1}\}$ and $\bar{w} = b_k$
Exchange Lemma $\Rightarrow$ vector $\bar{w}$ in S_{k-1} so
 $\text{Span}(S_{k-1} \cup \{b_k\} - \{\bar{w}\}) = \text{Span } S_{k-1}$

Set $S_k = S_{k-1} \cup \{b_k\} - \{\bar{w}\}$

Rank-Nullity Proof

$f(x) = Ax$
 A is an invertible matrix iff f is an invertible function
The function is invertible
iff $\dim \ker f = 0$ and $\dim \mathbb{R}^n = \dim \mathbb{R}^n$
iff $\text{nullity } A = 0$ and $|C| = |R|$ (square)
nullity $A = 0$ iff $\dim \text{Null } A = 0$
iff $\text{Null } A = \{0\}$
iff only vector x so $Ax = 0$ is $x=0$
iff columns A are linearly independent

Five Engine Lemma

For any $\bar{b}$ and $\bar{a}$ over the reals
 $\Rightarrow$ The projection $\bar{b}^a$ of $\bar{b}$ onto $\text{Span}\{e_3\}$ is the closest to $\bar{b}$
 $\Rightarrow$ Distance between $\bar{b}$ and $\bar{b}^a$ is $\|\bar{b}^{\perp a}\|$

Invertible functions one-to-one

Def: f is one-to-one if $f(x) = f(y) \Rightarrow x=y$
Proof: Assume an invertible func f is not one-to-one. So there exist $x_1 \neq x_2$ where $f(x_1) = f(x_2) = y$. Then $f^{-1}(y) = x_1$, but $f^{-1}(y) = x_2$ both cannot be true, by def of function

Invertible functions onto

func is onto if for every $z \in \mathbb{R}$ there exists element $a \in \mathbb{D}$ so $f(a) = z$
Proof: Assume invertible func f is not onto. So $\bar{z}$ in co-domain so that for no x does $f(x) = \bar{z}$. But $f^{-1}(\bar{z}) = x$ for some x , and by def of inverse $f(x) = \bar{z}$ which is a contradiction

Functional Invertibility Theorem

every invertible function is one-to-one and onto as proved above

Some SVD Lemma

Minimum Sum of squared

distance is $\|A\|_F^2 - \sigma_1^2$

def find_subset_basis($\{v_1, \dots, v_n\}$):

$\{v_1, \dots, v_n\}$ = orthogonalize($\{v_1, \dots, v_n\}$)
Return $[v_i \text{ for } i \text{ in } \{1, \dots, n\} \text{ if } v_i \text{ is not the zero vector}]$

def orthogonalize(v list):

$v_star_list = []$
for v in v_list :
 $v_star_list.append(\text{project_orthogonal}(v, v_star_list))$
return v_star_list

def project_orthogonal(b, v list):

for v in v_list :
 $b = b - \text{project_along}(b, v)$
return b

def project_along(b, v , eps = $1e-20$)

$sign = (b \cdot v) / (v \cdot v)$ if $v \cdot v > \text{eps}$ else 0
return $signs * v$

def morph(S, B)

$-S$: list of distinct vecs
 $-B$: list of linearly independent vecs in $\text{Span}(S)$
output: list of pairs of vecs to inject and eject

$b = \text{subset_basis}(S)$
used = $[False \text{ for } i \text{ in range}(\text{len}(b))]$
 $ans = []$

$i = 0$
while $i < \text{len}(B)$:
 if not used $[i]$:
 $b[i], B[-1] = B[-1], b[i]$
 if is_independent(b):
 $ans.append([b[i], B[-1]])$
 $B.pop()$
 used $[i] = True$
 else:
 $b[i], B[-1] = B[-1], b[i]$
 $i += 1$
 if $i \% \text{len}(b) == 0$:
 return ans

def trolly_line_location(A):

Given matrix A find vec $\bar{v}_1$ minimizing $\sum_i (\text{distance from row } i \text{ of } A \text{ to } \text{Span}\{\bar{v}_1\})^2$
 $\bar{v}_1 = \arg \max \{ \|A \cdot v\| : \|v\| = 1 \}$
 $\sigma_1 = \|A v_1\|$
return v_1

$\|A\|_F^2 - \|A \bar{v}\|^2 = \sum_i (\text{distance from row } i \text{ to } \text{Span}\{\bar{v}\})^2$

def is_eigenpair(A, v, lam)

return $v \neq \text{Vec}(v, 0)$ and $A \cdot v = lam \cdot v$

def find_eigen_values_vectors(A):

A : 2×2 matrix
1: sort row and column labels of A
2: get four entries a, b, c, d from A
3: compute trace $a+d$
4: determinant $ad-bc$
5: $D = (a+d)^2 - 4 \cdot \text{determinant}$
6: if D is negative return empty list
7: compute $\lambda_i = (a+d) \pm \sqrt{D} / 2$
8: Build identity matrix over column
9: if eigen values are different
 - $\text{eigenvalue_list} = [\lambda_1, \lambda_2]$
 - $\text{eigenvectors_list} = []$
 - for λ in eigenvalue list:
 $\text{nullspace}(A - \lambda \cdot \text{identity})$
 eigenvectors.append(nullspace)
 - return eigenvectors, eigenvectors
10: else $\lambda = \lambda_1$ and $\text{nullspace}(A - \lambda \cdot \text{id})$
 eigenvalues = $[\lambda] \times \text{len}(\text{nullspace})$
 eigen vectors = $\text{nullspace}[0]$
 return eigenvalues, vectors

def is_superfluous(L, i):

L : list of vecs
 i : int in range of L
Output: True if span of vecs are span excluding $L[i]$
if $\text{len}(L) == 1$:
 return $L[i] == \text{Vec}([0], 0, 0)$
 $A = \text{colDict } 2 \text{ mat}(L)$ if $i == 0$
 $b = L[i]$
 $x = \text{solve}(A, b)$
 $\text{residual} = A \cdot x - b$
if $\text{residual} \neq 0$:
 return True
else False

def is_independent(L)

for i in range($\text{len}(L)$):
 if is_superfluous(L, i):
 return False

def subset_basis(T)

$S = T[0:]$
 $i = 0$
while $i < \text{len}(S)$:
 if is_superfluous(S, i):
 $S.pop(i)$
 else $i += 1$
return S

def exchange(S, A, z)

S : list[vec]
 A : list[vec] $z = \text{vec}$
Output: vec w in S but not in A such $\text{Span}(S) = \text{Span}(\{z\} \cup S) - \{w\}$
for w in A :
 if w in A : continue
 $\text{new_set} = [z] + [v \text{ for } v \text{ in } S \text{ if } v \neq w]$
 if in span($\text{new_set}, w$)
 return w

def norm_f(A):

$A = \text{mat } 2 \text{ colDict}(A)$
 $\text{norm} = 0$
for v in range($\text{len}(A)$):
 $\text{norm} = v \cdot A[v]$
 $\text{norm} += \text{norm} \cdot v$
return $\sqrt{\text{norm}}$

def norm(v : vec)

return $(v \cdot v) ** 0.5$

def normalize(v : vec)

$n = \text{norm}(v)$
return $\frac{1}{n} \cdot v$ if $n \neq 0$
else $\text{Vec}(v, 0, 0)$

def projection(u, v)

find part of u that points in direction v
 $\text{denom} = v \cdot v$
if $\text{denom} == 0$:
 return $\text{Vec}(v, 0, 0)$
return $(\frac{u \cdot v}{\text{denom}}) \cdot v$

def rank_2_approx(u, z, v)

$u1 = \text{mat } 2 \text{ col}(u)[0]$
 $v1 = \text{mat } 2 \text{ col}(v)[0]$
 $\text{sigma} = S[0, 0]$
 $u1_col = \text{col } 2 \text{ mat}(\{u, u2\})$
 $v1_row = \text{col } 2 \text{ mat}(\{v, v1\})$
return $\text{sigma} * (u1_col + v1_row)$

def orthogonalComplement(v, w)

v : list-vec form basis span(v)
 w : list-vec form basis span(w)
Output: list of vec that for basis span(u) so span(u) is orthogonal complement of span(v) and reject span(w)
 $L = \text{orthogonalize}(v + w)$
 $u = []$
for vector u in L :
 if u is not zero vec
 $u.append(u)$
return u